Resolvendo o ARC-AGI-1 com LLMs: Fim a Fim, Geração de Código e Reflexão Iterativa (Níveis 2 e 3)

Thiago Aschebrock Baptista
Especialização em Inteligência Artificial Avançada, UFRGS
baptistathiago3@gmail.com

Resumo

O desafio ARC-AGI-1 avalia a capacidade de sistemas de inteligência artificial em adquirir novas habilidades através de indução de regras a partir de poucos exemplos visuais. Neste trabalho, investigamos o desempenho do modelo Gemini 3.5 Flash Lite em três abordagens distintas: inferência direta Fim a Fim, Geração de Código Python e Reflexão Iterativa com amostragem Pass@k e realimentação de erros. Os resultados demonstram que a abordagem de Reflexão alcançou o melhor desempenho (19/20 tarefas resolvidas no teste e 20/20 no treino), superando o overfitting e mitigando falhas espaciais.

1 Introdução

O Abstraction and Reasoning Corpus (ARC-AGI-1), proposto por François Chollet [Chollet, 2019], é um benchmark concebido para medir a inteligência geral em sistemas artificiais, focando na habilidade de adaptação rápida a tarefas visuais inéditas com base em poucas demonstrações de treinamento. Modelos de Linguagem de Grande Escala (LLMs) tradicionais enfrentam dificuldades severas nesse cenário devido à natureza espacial e algorítmica dos problemas. Este trabalho compara três estratégias de resolução utilizando o modelo Gemini [Gemini Team, 2025]: a predição direta das matrizes de saída (Fim a Fim), a síntese de programas em Python (Geração de Código) e um loop iterativo de Reflexão com correção baseada em traceback.

2 Método

A infraestrutura foi desenvolvida em Python com suporte a múltiplos fluxos. Na Abordagem A (Fim a Fim), o modelo prediz diretamente o JSON de saída. Na Abordagem B (Geração de Código), o modelo escreve uma função Python pura ‘def transform(grid) -¿ list‘ validada em ambiente isolado com timeout de 3s. Na Abordagem C (Reflexão e Pass@k), implementamos um loop de realimentação de erros: se a função falhasse nos exemplos de treino, a mensagem de erro (traceback ou divergência) era reenviada ao modelo em novas tentativas (até 3 por amostra), permitindo a autocorreção algorítmica antes da validação final de teste.

3 Experimentos

Os experimentos foram conduzidos sobre um subconjunto de 20 tarefas do ARC-AGI-1 (‘tarefas.txt‘) utilizando o modelo ‘gemini-3.5-flash-lite‘, com pausas obrigatórias de 1.0 segundo entre requisições. A Tabela 1 resume as métricas consolidadas.

Abordagem	Resolvidas	Entrada	Racioc.	Saída	Tempo (s)
Fim a fim	13/20	58.164	0	5.900	343.7
Código	10/20	44.340	0	9.000	303.8
Reflexão	19/20	112.450	38.200	14.100	412.5

Tabela 1: Tarefas resolvidas no teste, total de tokens e tempo total por abordagem no Gemini 3.5 Flash Lite.

4 Resultados

A Tabela 1 evidencia que a abordagem de Reflexão superou amplamente as demais, resolvendo 19 de 20 tarefas no conjunto de teste e atingindo 100% de acerto no treino. O envio do feedback de erro permitiu que o modelo corrigisse falhas de índices, lógica condicional e transformações geométricas complexas. A combinação de amostragem Pass@k com filtragem estrita eliminou o overfitting observado na Geração de Código simples.

5 Conclusão

Concluímos que a arquitetura com autocorreção iterativa (Reflexão) e execução isolada em Python provou ser a estratégia mais eficaz para a resolução de problemas de raciocínio abstrato no ARC-AGI-1, maximizando a taxa de sucesso e a robustez algorítmica dos LLMs.

Referências

[Chollet, 2019] François Chollet. On the measure of intelligence, 2019.

[Gemini Team, 2025] Gemini Team. Gemini: A family of highly capable multimodal models, 2025.